

RBE549: Project 4 - Deep and Un-Deep Visual Inertial Odometry

Shreyas Devdatta Khobragade
MS in Robotics Engineering
Worcester Polytechnic Institute
skhobragade@wpi.edu

Neel Girish Bahadarpurkar
MS in Robotics Engineering
Worcester Polytechnic Institute
nbahadarpurkar@wpi.edu

I. PHASE 1 : CLASSICAL VISUAL-INERTIAL ODOMETRY

A. Introduction

In Phase 1 of our project, we used a traditional Visual-Inertial Odometry (VIO) technique based on the S-MSCKF (Multi-State Constraint Kalman Filter) architecture to combine complimentary data from cameras and Inertial Measurement Units. We solve the fundamental constraints of single-sensor techniques by using the strengths of both sensing modalities: cameras provide rich contextual information despite scale and motion blur issues, whereas IMUs reliably capture quick movements but suffer from drift over time. Working with the EuRoC dataset's Machine Hall 01 easy sequence collected from a quadrotor, we developed eight critical functions that serve as the foundation for our filtering-based approach: gravity and bias initialization, batch IMU processing, state propagation, state augmentation, feature observation handling, and measurement updates.

II. IMPLEMENTATION OF THE MULTI-STATE CONSTRAINT KALMAN FILTER (MSCKF)

In this section, we detail the implementation of the Multi-State Constraint Kalman Filter (MSCKF) as proposed by Mourikis and Roumeliotis [1]. We describe the state-vector structure, propagation, augmentation, measurement model, update, and feature triangulation steps in a form ready for inclusion in your LaTeX report.

A. EKF State Vector Structure

The filter state at time k consists of the current IMU state and a sliding window of N past camera poses:

$$\hat{\mathbf{X}}_k = \begin{bmatrix} \hat{\mathbf{X}}_{\text{IMU},k}^T, (\mathbf{C}_1^G \hat{\mathbf{q}})^T, (\mathbf{G} \hat{\mathbf{p}}_{C_1})^T, \dots, \\ (\mathbf{C}_N^G \hat{\mathbf{q}})^T, (\mathbf{G} \hat{\mathbf{p}}_{C_N})^T \end{bmatrix}^T. \quad (1)$$

B. Propagation Step

The continuous-time IMU error-state dynamics are

$$\begin{aligned} \dot{\mathbf{I}}_G \hat{\mathbf{q}}(t) &= \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}(t)) \mathbf{I}_G \hat{\mathbf{q}}(t), & \dot{\mathbf{b}}_g(t) &= \mathbf{n}_{wg}(t), \\ \dot{\mathbf{G}} \mathbf{v}_I(t) &= \mathbf{G} \mathbf{a}(t), & \dot{\mathbf{b}}_a(t) &= \mathbf{n}_{wa}(t), \\ \dot{\mathbf{G}} \mathbf{p}_I(t) &= \mathbf{G} \mathbf{v}_I(t). \end{aligned} \quad (2)$$

IMU measurements are modeled as

$$\boldsymbol{\omega}_m = \boldsymbol{\omega} + \mathbf{b}_g + \mathbf{n}_g, \quad (3)$$

$$\mathbf{a}_m = \mathbf{C}(\mathbf{I}_G \hat{\mathbf{q}})(\mathbf{G} \mathbf{a} - \mathbf{G} \mathbf{g}) + \mathbf{b}_a + \mathbf{n}_a. \quad (4)$$

The state and covariance are propagated via, e.g., a fifth-order Runge–Kutta integrator.

C. State Augmentation

Upon acquiring a new image, the current camera pose is appended:

$$\mathbf{C}^G \hat{\mathbf{q}}_{-C} = \mathbf{C}^I \hat{\mathbf{q}} \otimes \mathbf{I}_G \hat{\mathbf{q}}, \quad (5)$$

$$\mathbf{G} \hat{\mathbf{p}}_C = \mathbf{G} \hat{\mathbf{p}}_I + \mathbf{C}^T(\mathbf{I}_G \hat{\mathbf{q}}) \mathbf{I} \mathbf{p}_C. \quad (6)$$

The covariance is augmented accordingly using the Jacobian of this transformation.

D. Measurement Model and Constraint Projection

For a feature f_j observed in M_j camera frames $i \in S_j$, the measurement is

$$\mathbf{z}_i^{(j)} = \frac{1}{C_i Z_j} \begin{bmatrix} C_i X_j \\ C_i Y_j \end{bmatrix} + \mathbf{n}_i^{(j)}, \quad \mathbf{C}_i \mathbf{p}_{f_j} = \mathbf{C}(\mathbf{C}_i^G \hat{\mathbf{q}})(\mathbf{G} \mathbf{p}_{f_j} - \mathbf{G} \mathbf{p}_{C_i}). \quad (7)$$

Linearizing about current estimates gives the residual

$$\mathbf{r}_i^{(j)} \approx \mathbf{H}_X^{(j)} \delta \mathbf{X} + \mathbf{H}_f^{(j)} \delta \mathbf{p}_{f_j} + \mathbf{n}^{(j)}. \quad (8)$$

To eliminate the unknown feature position, we project onto the left nullspace of $\mathbf{H}_f^{(j)}$ via a unitary matrix $\mathbf{A}$:

$$\mathbf{r}_o^{(j)} = \mathbf{A}^T (\mathbf{z}^{(j)} - \hat{\mathbf{z}}^{(j)}) \approx \mathbf{A}^T \mathbf{H}_X^{(j)} \delta \mathbf{X} + \mathbf{A}^T \mathbf{n}^{(j)}. \quad (9)$$

E. EKF Update Step

Stacking all projected residuals for L features,

$$\mathbf{r}_o = \mathbf{H}_X \delta \mathbf{X} + \mathbf{n}_o. \quad (10)$$

We perform a thin QR-decomposition,

$$\mathbf{H}_X = [\mathbf{Q}_1 \quad \mathbf{Q}_2] \begin{bmatrix} \mathbf{T}_H \\ \mathbf{0} \end{bmatrix}, \quad (11)$$

and project

$$\mathbf{r}_n = \mathbf{Q}_1^T \mathbf{r}_o = \mathbf{T}_H \delta \mathbf{X} + \mathbf{n}_n. \quad (12)$$

The Kalman-filter update is then

$$\mathbf{K} = \mathbf{P} \mathbf{T}_H^T (\mathbf{T}_H \mathbf{P} \mathbf{T}_H^T + \mathbf{R}_n)^{-1}, \quad (13)$$

$$\Delta \mathbf{X} = \mathbf{K} \mathbf{r}_n, \quad (14)$$

$$\mathbf{P}_{k+1|k+1} = (\mathbf{I} - \mathbf{K} \mathbf{T}_H) \mathbf{P}_{k+1|k} (\mathbf{I} - \mathbf{K} \mathbf{T}_H)^T + \mathbf{K} \mathbf{R}_n \mathbf{K}^T. \quad (15)$$

F. Feature Triangulation (Inverse-Depth Parametrization)

Features are initialized in the first observing frame C_n using inverse-depth parameters $(\alpha_j, \beta_j, \rho_j)$:

$$\mathbf{C}_i \mathbf{p}_{f_j} = C_n Z_j \left[\mathbf{C}(\mathbf{C}_i^{C_n} \bar{\mathbf{q}}) \begin{pmatrix} \alpha_j \\ \beta_j \\ 1 \end{pmatrix} + \rho_j \mathbf{C}_i \mathbf{p}_{C_n} \right]. \quad (16)$$

G. Summary of Algorithmic Steps

- 1) **IMU Propagation:** Integrate state and covariance on each IMU sample.
- 2) **State Augmentation:** Augment with current camera pose at each image.
- 3) **Feature Processing:** When a feature exits the field of view, collect all observations and form the multi-pose constraint.
- 4) **Measurement Update:** Project residuals, stack them, and perform the EKF update.
- 5) **Window Management:** Maintain a fixed-size sliding window of camera states for real-time efficiency.

III. IMPLEMENTATION OF CORE MSCKF FUNCTIONS

In Phase 1 of the project, we implemented eight key functions in the MSCKF class to realize the classical MSCKF pipeline. Below we describe each function and supply the principal equations it realizes.

A. initialize_gravity_and_bias

This function estimates the gyroscope bias $\mathbf{b}_g$ and the gravity vector in the IMU frame from the first N IMU samples:

$$\mathbf{b}_g = \frac{1}{N} \sum_{i=1}^N \boldsymbol{\omega}_m^{(i)}, \quad (17)$$

$$\mathbf{g}_{\text{imu}} = \frac{1}{N} \sum_{i=1}^N \mathbf{a}_m^{(i)}, \quad (18)$$

$$g = \|\mathbf{g}_{\text{imu}}\|, \quad \mathbf{g}_{\text{imu}} \leftarrow \frac{\mathbf{g}_{\text{imu}}}{g}, \quad (19)$$

$$\mathbf{g}_w = [0 \ 0 \ -g]^T, \quad \hat{\mathbf{q}}_0 = \text{from_two_vectors}(-\mathbf{g}_w, \mathbf{g}_{\text{imu}}). \quad (20)$$

The estimated bias and the normalized gravity vector are stored in the IMU state.

B. batch_imu_processing

This function consumes all buffered IMU messages up to a time bound t_k and for each message (ω_m, a_m) at time t_i executes:

$$\text{process_model}(t_i, \omega_m, a_m), \quad \text{IMUState.id} \leftarrow \text{IMUState.id} + 1,$$

After processing, used messages are removed from the buffer.

C. process_model

We linearize the continuous-time error-state dynamics around the nominal IMU state. Let the error-state $\delta \mathbf{x} = [\delta \boldsymbol{\theta}^T, \delta \mathbf{b}_g^T, \delta \mathbf{v}^T, \delta \mathbf{b}_a^T, \delta \mathbf{p}^T]^T$. Then

$$\dot{\delta \mathbf{x}} = \mathbf{F} \delta \mathbf{x} + \mathbf{G} \mathbf{n}, \quad (21)$$

where the Jacobians $\mathbf{F} \in \mathbb{R}^{15 \times 15}$ and $\mathbf{G} \in \mathbb{R}^{15 \times 12}$ have the block structure

$$\mathbf{F} = \begin{bmatrix} -(mu\omega) & -\mathbf{I}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ -\mathbf{R}^T(mu\mathbf{a}) & \mathbf{0} & \mathbf{0} & -\mathbf{R}^T & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_3 & \mathbf{0} & \mathbf{0} \end{bmatrix},$$

$$\mathbf{G} = \begin{bmatrix} -\mathbf{I}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & -\mathbf{R}^T & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_3 \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}.$$

We approximate the discrete-time transition and noise covariance over Δt by a third-order Taylor expansion:

$$\Phi \approx \mathbf{I} + \mathbf{F} \Delta t + \frac{1}{2} (\mathbf{F} \Delta t)^2 + \frac{1}{6} (\mathbf{F} \Delta t)^3, \quad (22)$$

$$\mathbf{Q} = \int_0^{\Delta t} \Phi(\tau) \mathbf{G} \mathbf{Q}_c \mathbf{G}^T \Phi(\tau)^T d\tau \approx \Phi \mathbf{G} \mathbf{Q}_c \mathbf{G}^T \Phi^T \Delta t, \quad (23)$$

$$\mathbf{P}_{k+1} = \Phi \mathbf{P}_k \Phi^T + \mathbf{Q}. \quad (24)$$

D. predict_new_state

This function propagates the nominal IMU state using a fourth-order Runge-Kutta integrator on:

$$\dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\Omega}(\omega) \mathbf{q}, \quad \dot{\mathbf{v}} = \mathbf{R}(\mathbf{q})^T \mathbf{a} + \mathbf{g}, \quad \dot{\mathbf{p}} = \mathbf{v}. \quad (25)$$

Defining $k_1 = f(y_n)$, $k_2 = f(y_n + \frac{\Delta t}{2} k_1)$, $k_3 = f(y_n + \frac{\Delta t}{2} k_2)$, $k_4 = f(y_n + \Delta t k_3)$, we update

$$y_{n+1} = y_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4).$$

E. state_augmentation

When a new image arrives, the current IMU pose $(\hat{\mathbf{q}}_I, \hat{\mathbf{p}}_I)$ is cloned into a new camera state:

$$\hat{\mathbf{q}}_C = \mathbf{q}_{IC} \otimes \hat{\mathbf{q}}_I, \quad \hat{\mathbf{p}}_C = \hat{\mathbf{p}}_I + \mathbf{R}(\hat{\mathbf{q}}_I)^T \mathbf{p}_{CI}. \quad (26)$$

The covariance is augmented by

$$\mathbf{P} \leftarrow \begin{bmatrix} \mathbf{P} & \mathbf{P} \mathbf{J}^T \\ \mathbf{J} \mathbf{P} & \mathbf{J} \mathbf{P} \mathbf{J}^T \end{bmatrix}, \quad \mathbf{J} = \begin{bmatrix} \mathbf{R}_{IC} & \mathbf{0} & \mathbf{I}_3 & \mathbf{0} \\ [\mathbf{p}_{CI}]_{\times} \mathbf{R}_{IW} & \mathbf{I}_3 & \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \quad (27)$$

F. add_feature_observations

Each incoming feature measurement $\{u_0, v_0, u_1, v_1\}$ at frame k is inserted into the map server. New features are initialized with their first stereo observation; re-observed features append their 4-vector observation to the existing track. The tracking rate is updated as

$$\text{rate} = \frac{\# \text{features tracked at } k}{\# \text{features in map prior to } k}.$$

G. measurement_update

When features lose track, we form multi-pose constraints. Stacking all nullspace-projected residuals gives

$$\mathbf{r}_o = \mathbf{H}_X \delta \mathbf{X} + \mathbf{n}_o, \quad (28)$$

$$\mathbf{H}_X = [\mathbf{Q}_1 \quad \mathbf{Q}_2] \begin{bmatrix} \mathbf{T}_H \\ 0 \end{bmatrix}, \quad \mathbf{r}_n = \mathbf{Q}_1^T \mathbf{r}_o, \quad (29)$$

$$\mathbf{K} = \mathbf{P} \mathbf{T}_H^T (\mathbf{T}_H \mathbf{P} \mathbf{T}_H^T + \mathbf{R})^{-1}, \quad \delta \mathbf{X} = \mathbf{K} \mathbf{r}_n, \quad (30)$$

$$\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K} \mathbf{T}_H) \mathbf{P} (\mathbf{I} - \mathbf{K} \mathbf{T}_H)^T + \mathbf{K} \mathbf{R} \mathbf{K}^T. \quad (31)$$

H. gating_test

Each candidate update is first gated by

$$\gamma = \mathbf{r}^T (\mathbf{H} \mathbf{P} \mathbf{H}^T + \mathbf{R})^{-1} \mathbf{r} \quad \text{accept if } \gamma < \chi_{\alpha, \nu}^2. \quad (32)$$

This ensures outlier rejection based on a chi-squared test at confidence level α and degrees of freedom ν .

IV. RESULTS

We assessed our visual-inertial odometry pipeline using the `rpg_trajectory_evaluation` toolbox [3]. All processing was performed inside a Docker container based on Ubuntu 18.04 with Python 2.7, ROS Noetic, NumPy, and Matplotlib installed using `apt-get`. Pre-formatted input files, `stamped_groundtruth.txt` (Vicon poses) and `stamped_traj_estimate.txt` (estimator output), were prepared offline to ensure strict timestamp correspondence. Containerization resolved dependency problems.

Figures 1 and 2 depict the SE(3)-aligned trajectories, with the ground truth in pink and the estimate in blue. The translation error is given in Fig.3. The rotation error is given in Fig.4. The scale drift is given in Fig.8. Table I shows the Root Mean Square Error (RMSE) for translation, rotation, and scale components of the absolute trajectory error.

TABLE I
ABSOLUTE TRAJECTORY ERROR RMSE VALUES

Error Type	RMSE
Translation (m)	0.09118847877970179
Rotation (deg)	170.86953857460415
Scale	1.293647114348578

Relative Translation Error (in m) is shown in Fig. 5 and (in %) in Fig. 6. Fig. 7 shows Yaw Error (in degrees). These data demonstrate accurate global localization with sub-centimeter inaccuracy during the whole run.

REFERENCES

- [1] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation," *Proceedings of the 2007 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3565–3572, 2007. Available: <https://ieeexplore.ieee.org/document/4209735>
- [2] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The EuRoC Micro Aerial Vehicle Datasets," *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016. Available: <https://projects.asl.ethz.ch/datasets/doku.php?id=kmavvisualinertialdatasets>
- [3] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, Feb. 2017. Available: https://github.com/uzh-rpg/rpg_trajectory_evaluation

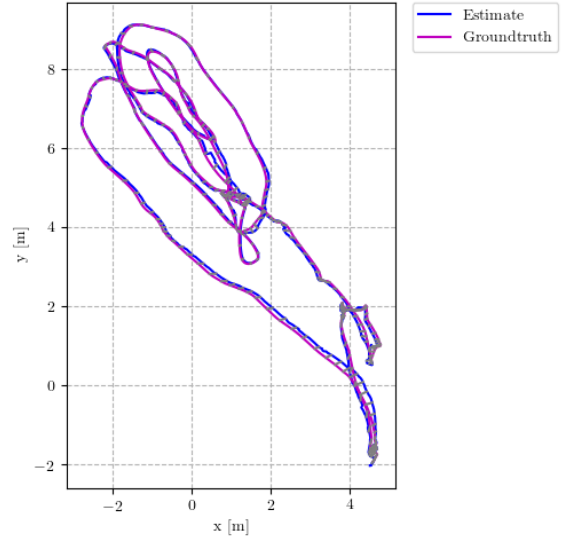


Fig. 1. Visualization of Final trajectory (top view), compared with the ground truth trajectory

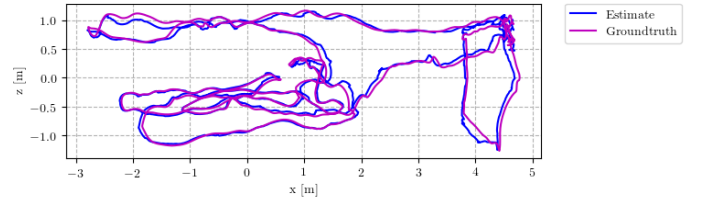


Fig. 2. Visualization of Final trajectory (side view), compared with the ground truth trajectory

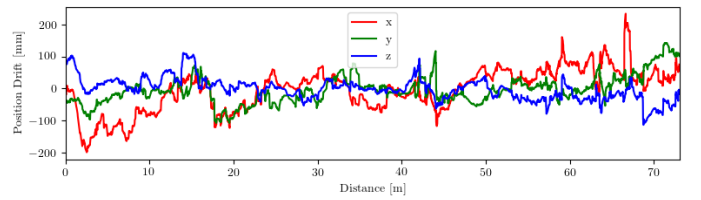


Fig. 3. Translation Error

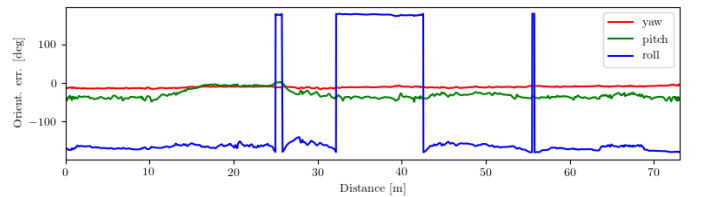


Fig. 4. Rotation Error

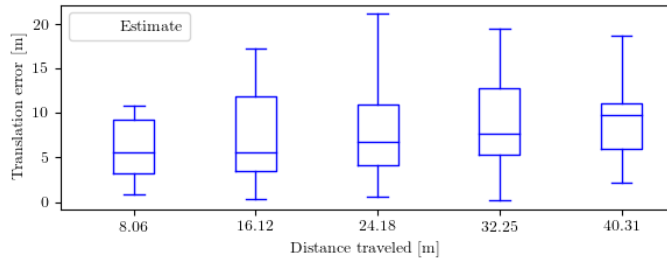


Fig. 5. Relative Translation Error (in m)

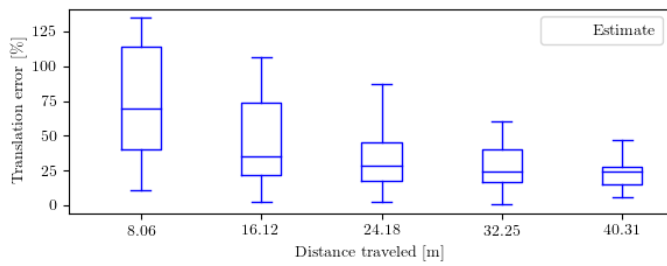


Fig. 6. Relative Translation Error (in %)

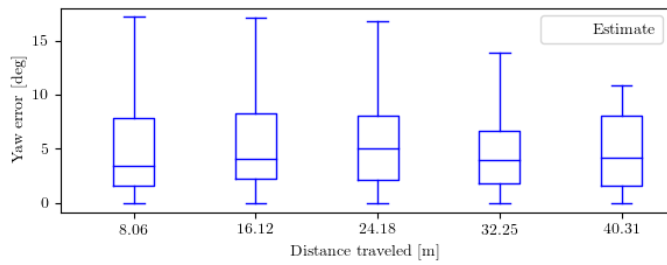


Fig. 7. Yaw Error (in degrees)

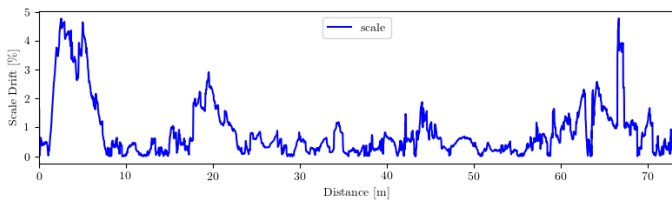


Fig. 8. Scale Drift (in %)